

Aula 25 — Sistemas Operacionais Distribuídos: Gerência de Processos e Escalonamento Distribuído

Disciplina: Sistemas Distribuídos — Uni-FACEF · Prof. Leandro Borges

Da Aula 24 para a Aula 25

Na Aula 24 conhecemos o conceito de Sistema Operacional Distribuído (SOD), a diferença entre um SO de Rede (NOS) e um SOD propriamente dito, os três modelos clássicos de arquitetura — workstation, processor pool e híbrido — e o papel do middleware em viabilizar, na prática, boa parte da transparência que um SOD puro prometeria entregar. Ao final daquela aula, listamos três desafios centrais de projeto de um SOD: gerência global de recursos, escalonamento distribuído e comunicação entre kernels. Esta aula aprofunda o segundo desses desafios — e, para chegar até ele, também precisa tratar de um problema relacionado: como processos e threads são geridos quando existem várias máquinas envolvidas, e o que significa mover um processo de uma máquina para outra em plena execução. O tema seguinte, sistemas de arquivos distribuídos, fica para a Aula 26.

Threads e processos em um sistema distribuído

Um processo é a unidade tradicional de execução de um sistema operacional: possui seu próprio espaço de endereçamento, seus próprios recursos (arquivos abertos, conexões de rede) e é isolado de outros processos. Essa proteção tem um custo — criar um processo novo e trocar de contexto entre processos são operações relativamente caras, porque envolvem manipular tabelas de memória inteiras. Uma thread é uma unidade de execução mais leve: várias threads podem existir dentro de um mesmo processo, compartilhando seu espaço de endereçamento e seus recursos abertos, mas cada uma mantém sua própria pilha de execução e seu próprio contador de programa. Como threads de um mesmo processo compartilham memória, criá-las e alternar entre elas é muito mais barato do que fazer o mesmo com processos completos.

Essa diferença de custo é especialmente relevante em sistemas distribuídos, porque um servidor típico — um servidor de arquivos, um servidor de aplicação, um servidor de banco de dados — precisa atender múltiplas requisições concorrentes vindas de clientes diferentes, muitas vezes ao mesmo tempo. Se cada requisição exigisse a criação de um processo novo, o servidor rapidamente se tornaria um gargalo. Threads leves permitem que o servidor ofereça paralelismo real entre as requisições sem pagar esse preço.

O modelo de thread por requisição

Um padrão de projeto muito comum para explorar essa vantagem é o modelo de thread por requisição (thread-per-request). Nesse modelo, o processo servidor mantém uma thread despachante, responsável apenas por aceitar novas conexões de clientes. Assim que uma nova requisição chega, a thread despachante cria (ou reaproveita, em variações com pool de threads) uma thread dedicada para atender aquele cliente específico, enquanto ela mesma volta imediatamente a aguardar novas conexões. Cada thread de atendimento executa de forma independente das demais: como compartilham o mesmo espaço de endereçamento do processo, todas têm acesso natural às mesmas estruturas de dados internas do servidor, mas a lógica de atendimento de um cliente não bloqueia o atendimento dos outros. É esse modelo, ou variações dele, que sustenta a capacidade de servidores distribuídos reais de atender centenas ou milhares de clientes simultaneamente com uso eficiente de CPU e memória.

Migração de processos: conceito e motivações

Migração de processos é a capacidade de transferir um processo já em execução de uma máquina para outra, preservando o estado que ele já acumulou — variáveis, pilha de execução, e o ponto exato do programa em que ele estava. É diferente de simplesmente iniciar uma nova instância do mesmo programa em outra máquina: na migração, o processo continua de onde parou, como se a mudança de máquina fosse invisível para ele.

Existem pelo menos três motivações práticas para migrar um processo. A primeira é o balanceamento de carga: se uma máquina está sobrecarregada enquanto outra está ociosa, mover processos da primeira para a segunda melhora o uso global dos recursos do sistema. A segunda é a proximidade dos dados: em certas aplicações, é mais eficiente levar a computação para perto de onde os dados que ela consome estão armazenados do que transportar grandes volumes de dados pela rede até onde o processo está rodando. A terceira é a manutenção do sistema: quando uma máquina precisa ser desligada para manutenção, atualização ou substituição, migrar os processos que nela rodavam para outras máquinas evita interromper o serviço que esses processos prestavam.

Migração de processos: os desafios técnicos

Apesar de conceitualmente simples, migrar um processo em execução é uma operação tecnicamente exigente, porque é preciso reconstruir fielmente, na máquina de destino, tudo o que o processo precisa para continuar rodando sem que ninguém perceba a mudança. Um primeiro desafio é transferir a memória do processo — todo o seu espaço de endereçamento, incluindo código, pilha e heap, precisa ser copiado (ou, em técnicas mais sofisticadas, copiado de forma incremental) para o nó de destino. Um segundo desafio é reconectar os recursos abertos: arquivos que o processo mantinha abertos e conexões de rede (sockets) que ele mantinha estabelecidas precisam continuar válidos depois da migração, o que normalmente exige coordenação com o sistema operacional de origem e de destino. Um terceiro desafio é redirecionar mensagens em trânsito — outros processos que já se comunicavam com o processo migrado, endereçando-o pela sua localização antiga, precisam ser capazes de alcançá-lo no novo endereço, sem perder mensagens que já estavam a caminho. Por fim, existe sempre uma pressão para minimizar o tempo de congelamento do processo durante a transferência: quanto mais tempo ele fica pausado enquanto seu estado é movido, maior é o impacto percebido por quem depende dele. Por causa desses desafios, migração de processos completa é usada de forma seletiva na prática, geralmente reservada a cenários em que os ganhos de balanceamento de carga ou de manutenção compensam claramente o custo da operação.

Escalaonamento distribuído: o que muda em relação ao escalaonamento local

Em um sistema operacional tradicional, escalaonar processos significa decidir qual processo, entre os que aguardam na fila de prontos, deve ocupar a CPU agora. É uma decisão local, tomada com informação completa e atualizada sobre o estado de uma única máquina. Em um sistema distribuído, essa pergunta continua existindo — cada máquina ainda precisa decidir, entre seus próprios processos, qual roda agora —, mas surge uma pergunta anterior e mais difícil: em qual máquina, entre todas as disponíveis na rede, esse processo deveria executar? Essa é a tarefa do escalaonamento distribuído, e ela é qualitativamente diferente do escalaonamento local. Enquanto o escalaonador local tem visão completa e de baixíssima latência sobre o estado da sua própria máquina, o escalaonador distribuído precisa decidir com base em informação de carga que vem de outras máquinas pela rede — informação que é, por natureza, mais lenta de obter e potencialmente desatualizada no momento em que a decisão é tomada. O escalaonamento distribuído acontece antes do escalaonamento local: primeiro se decide o destino do processo, depois o escalaonador daquela máquina específica decide quando, entre os processos que ali residem, cada um ocupa a CPU.

Estratégias de balanceamento de carga: estático e dinâmico

Para decidir em qual máquina colocar cada processo, os algoritmos de escalonamento distribuído recorrem a estratégias de balanceamento de carga, que se dividem, num primeiro eixo, entre estáticas e dinâmicas. Uma estratégia estática toma a decisão de alocação de forma fixa e antecipada — por exemplo, distribuindo processos entre as máquinas segundo uma regra predefinida ou em round-robin — sem observar a carga real de cada nó no momento da decisão. É simples de implementar e tem baixo overhead de execução, mas pode desperdiçar recursos se a carga efetiva das máquinas mudar ao longo do tempo de um jeito que a regra fixa não previu. Uma estratégia dinâmica, em contraste, toma a decisão com base na carga atual medida em cada máquina, reagindo a mudanças de carga conforme elas acontecem. Isso tende a produzir um uso mais eficiente dos recursos do sistema, mas exige que as máquinas monitorem sua própria carga e a troquem entre si (ou com um coordenador), o que introduz overhead de comunicação e de processamento que a estratégia estática não tem.

Quem inicia o balanceamento: sender-initiated e receiver-initiated

Dentro das estratégias dinâmicas, existe ainda uma segunda escolha de projeto: quem toma a iniciativa de agir quando o sistema está desbalanceado? Em uma abordagem sender-initiated (iniciada pelo remetente), é a máquina sobrecarregada que assume a iniciativa: ela percebe que sua carga está alta e passa a procurar ativamente por outra máquina mais ociosa para a qual possa transferir trabalho. Em uma abordagem receiver-initiated (iniciada pelo destinatário), é a máquina ociosa que toma a iniciativa: ela percebe que tem capacidade livre e passa a anunciar essa disponibilidade, ou a solicitar ativamente trabalho de outras máquinas mais carregadas. Cada abordagem tende a se sair melhor em cenários diferentes: estratégias sender-initiated funcionam bem quando a carga geral do sistema é baixa a moderada, já que a maioria das máquinas está ociosa e disposta a receber trabalho quando solicitada; estratégias receiver-initiated tendem a funcionar melhor quando a carga geral do sistema já é alta, porque nesse cenário há mais máquinas sobrecarregadas do que ociosas, e faz mais sentido que as poucas máquinas com capacidade livre procurem trabalho ativamente.

O papel dos algoritmos de eleição no escalonamento distribuído

Algumas arquiteturas de escalonamento distribuído dependem de um nó coordenador para centralizar decisões — por exemplo, reunindo periodicamente as informações de carga de todas as máquinas do sistema antes de decidir para onde migrar um processo. Esse desenho reintroduz um problema que já estudamos nas aulas de coordenação e consenso: o que fazer quando o coordenador falha ou precisa ser substituído? É exatamente para resolver esse problema que servem os algoritmos de eleição, como o algoritmo Bully, que já vimos em detalhe anteriormente. Quando o coordenador atual para de responder, os demais nós disparam uma eleição para escolher um novo coordenador de forma consistente, sem exigir intervenção manual, permitindo que o escalonamento distribuído continue funcionando mesmo diante de falhas. Não é necessário repetir aqui o funcionamento do algoritmo — o ponto importante é reconhecer que ele reaparece como peça de apoio sempre que um mecanismo distribuído, como o escalonamento, depende de um coordenador único.

Referências

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems: Principles and Paradigms. 3. ed. Pearson, 2017. Cap. 3 (Processos) — threads, migração de código e algoritmos de escalonamento.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. Distributed Systems: Concepts and Design. 5. ed. Addison-Wesley, 2011. Cap. 4 (Interação) e Cap. 6 (Coordenação e Acordo) — modelos de processo e coordenação distribuída.

TANENBAUM, A. S.; VAN STEEN, M. Distributed Systems. 4. ed. 2023 (edição digital gratuita) — capítulo sobre processos, threads e migração.